Assignment - 17

1] What is meant by function overloading
→ Refer Q.10 from Assignment - 13

2] Why function overloading is considered as compile time polymorphism
→ Function overloading allows multiple function in same scope to have the same name but with different parameter lists (different type, number or order of parameters).
Considered Compile time polymorphism:
1) Function to be called is determined by the compiler at compile time, based on function signature
2) There is no decision made at runtime, the compiler uses the argument types to resolve which func to invoke.

3] What do you mean by name mangling/decoration (naming)?
→ When we compile the code, the compiler changes the name of every function with 'managed name' 'mangled name'.
— Refer Q.5 Assignment - 14

4] Why return value is not considered as function overloading criteria.
→ Return type is not of function's signature used for overloading bcoz:
- When a funct^n is called, compiler determines the appropriate function based only on the arguments provided, not based of return type
- Depending on return type for disambiguation would make funct^n call ambiguous and lead to

complex rules for determining which function to invoke

5] Why & what is use of function overloading

→ A] Why:
- To enhance code readability & maintainability
- To allow same func name to perform similar operations for different types of data or scenarios.

B] Uses:
- Provides a way to define multiple behaviours under one function name
- Avoids the need for multiple distinct function names, which can be confusing

E.g. int add (int a, int b)
{ return a+b; }

double add (double a, double b)
{ return a+b; }

→ the add() function works seamlessly for both int & double types.

6] What are the scenarios in which we cannot perform function overloading?

→ 1) When functions differ only by Return type:
As discussed, return type is not part of signature used for overloading.

2) Ambiguous Overloads: overloading with default arguments can create ambiguity

void display (int a);
void display (int a, int b = 10); // ambiguous which to call for display (5)

3) Functions differing only by 'const':
Non-const & const versions of a function cannot be overloaded unless used in classes with 'this' pointer
Eg void print(int a) const; //only valid in member-func
void print(int a); // Non-const version

4) What are the scenarios in which we can overload the function?

→ 1) Different Parameter Types:
void print(int a);
void print(double a);

2) Diff no of parameters
void print(int a);
void print(int a, b);

3) Diff Parameter order:
void print(int a, double b);
void print(double a, int b);

4) With ref or Pointer Parameters:
void print(int &a);
void print(int *a);

Rules:-
1) Names of all functions should be same
2) Data types of parameters should be different
3) No of parameters should be different
4) We cannot overload function by changing its return value.

we can overload func by changing:
1) No. of Arguments.
2) Sequence of Arguments.
3) Data types of Arguments.

8] Predict the output of below program?
→ The above code demonstrates - function overloading where fun() is overloaded, using different parameter lists.
1) void fun (int i)
2) void fun (int i, int j)

- obj.fun(10); it matches the 1st version of fun() bcoz it accepts a single integer.
- obj.fun(10, 20); matches 2nd version of fun() because it has 2 parameters (integers).

- Output
1) when 1st funct^n is called it points → First definition
2) when 2nd — | ——— → Second —|—

Output : First definition
Second definition
[If endl used]

Actual : [as no endl]
First definitionSecond definition

9]
s func() overloaded 3 times:
1) void fun(int *p) - Accepts an integer pointer
2) void fun (float *p) - — | — float pointer
3) void fun (int no) - — | — integer (by value)

Calls:

1) obj.fun(no):
   - no is an integer
   - matches void fun(int no) → 3rd function
   - Prints "Third definition"

2) obj.fun(&no):
   - &no is a pointer to an int
   - matches void fun(int *p) → 1st fun
   - Prints "First definition"

3) obj.fun(&f):
   - &f is a pointer to a float
   - matches void fun(float *p) → 2nd fun
   - Prints "Second definition"

Output: Third definition First definition Second definition
(just coz there is nu endl)

10] Draw object layout & class diagram code snippets, explain internal workings. Explain type of inheritance

⇒ Classes & Members.

1] Class Base:
Non-static data member: i, j,
static ——— : k
Constructor: Initializes → i=10, j=20
Member function: void fun(), prints "Base fun"

2] Class Derived:
Inherits Base publicly (public access) means Base's public & protected members are accessible as public & protected in Derived
Adds new data members: x, y

Constructor initialize: x = 100, y = 200

member function → void fun();

Types of Inheritance

1) Public - Derived class inherits all public & and single protected member of Base class

Memory Layout		
member	Type	Value (after construction)
i	int	10
j	int	20
k	static int	12 (shared by all objects)
Base		
Base::i	int	10
Base::j	int	20
Derived::x	int	100
Derived::y	int	200
Base::k	static int	12
Derived		

Object Layout

dobj

100	10	Base::i
104	20	Base::j
108	12	Base::k
112		

dobj

200	10	Base::i
204	20	Base::j
208	100	Derived::i
212	200	Derived::j
216	12	Base::k
220		

- Static member - k is a class level variable & shared accross all objects of Base & Derived.
- Derived class inherits (all beh & chars) of Base class ('Single Inheritance')
- Derived object created:

1) Base Constructor is called 1st to initialize i & j
2) Derived — " — 2nd to initialize x & y.